EASILE 2D Anti-Cheat Game

PATCH-02 | DLL Injection 공격·방어 실험 보고서

실험일 / 작성자	2026-10-9 / 정명훈
대상 / 빌드	GameClient.exe / Release
Baseline	PATCH-01 방어 적용 빌드
공격 목표	DLL Injection으로 플레이어의 이동 속도를 2배 빠르게 하는 것이 목적이다.
최종 결과	공격 성공	방어 진행중

1. 실험 환경 및 정상 상태

OS / 도구	Windows 11 x64 / IDA Free 9.3, x64dbg64, Visual Studio 2022
대상 EXE SHA-256	0x17B41D48710F1BCF83CE120AD8871EAB765060BA80B6071CF313FF53BA9CF38F
주입 DLL SHA-256	
정상 동작 기준	궁극기 10초 쿨타임 / 이동·공격·Bot 정상 / 기존 테스트 PASS
기존 방어	GameWorld::Update() CRC32 검사 + Runtime Cooldown Validation

2. 공격 설계 및 분석

공격 기법	LoadLibrary 기반 공격
대상 프로세스 / DLL	Build?Release/GameClient.exe / payload.dll
주입 지점 및 방식	kernel32.dll을 통해서 주입을 할 예정이다.
DLL 내부 변경 목표	플레이어의 이동속도를 2배 증가시키려고 한다.

핵심 주소·함수·근거	hero구조체에 존재하는 speed값 변수
	00000038 int ultimateDamage_;
	0000003C float speed_;
	00000040 long double ultimateCd_;
	00000048 long double reveal_;
	GameWorld *__fastcall GameWorld::GameWorld(GameWo
	{
	this->player_.spawn_.x = 205.0;
	this->player_.__vftable = (PlayerHero_vtbl *)Pl
	this->player_.team = Blue;
	12 this->player_.requiredXp_ = 100;
	13 this->player_.basicDamage_ = 5;
	14 this->player_.ultimateDamage_ = 30;
	15 this->player_.speed_ = 220.0;
	16 this->player_.ultimateCd_ = 0.0;
	17 this->player_.reveal_ = 0.0;
	gameworld함수에서 플레이어의 스피드값을 설정하고 있음을 확인해 이 함수가 핵심함수임을 알 수 있었다.

탐색 과정 (단서 → 분석 → 최종 판단)

우선적으로는 dll_injection을 하기 위해서 먼저 기존의 dll_injection에서 배웠던 내용에 대해서 복습을 했다. 그 다음에 이전에 수업에서 이용했던 injector.exe파일과 코드를 release폴더에 옮겨 놓고 그 다음에 payload.c파일까지 만들어서 dll injection을 위한 기초적인 준비는 다 맞췄다. 그 다음에는 codex에게 현재 적용된 방어 기법 그리고 시간을 고려해서 현재 dll injection을 할 만한 후보를 추천해 달라고 했고 codex는 후보로 hp,이동 속도,기본 공격력 피해량을 제시했고 나는 그 중에서 가장 시각적으로 보여주기 좋은 이동 속도를 선택했다.

3. 공격 수행 및 검증

단계	수행 내용 / 관찰 결과
1	IDA를 통해 Hero 구조체의 speed_초기화 위치(RVA 0x10DE8)와 기본값 220.0f확인
2	이동속도를 440.0f로 변경하는 payload.dll제작 및 Release x64 빌드
3	injector2.exe를 통해 게임 프로세스에 DLL 주입 시도 후 이동속도 변화 확인

검증 항목	주입 전	주입 후
DLL 로드 / 주입 성공	해당 없음	payload.dll주입 성공
목표 동작 변화	핵이 막힌 상태에서 기본 공격만 사용 가능하고 나머지는 기존과 동일	속도가 2배 빨라짐
게임 정상 실행	☐ 정상	정상
기존 방어 탐지 여부	궁극기 관련 탐지함	미탐지

-

공격 판정: 성공

4. 원인 분석 및 방어 설계

공격이 가능했던 이유 / 기존 CRC32·런타임 검사의 탐지 범위와 한계

기존 방어는 GameWorld::Update() 함수의 특정 코드 영역에 대한 CRC32 무결성 검사와 궁극기 사용 간격을 검증하는 런타임 검사를 적용하였다.

그러나 이번 공격은 해당 검사 범위 밖에 위치한 Hero의 이동속도 초기화 명령어(RVA 0x10DE8)를 수정하였다. DLL 인젝션을 통해 기존 이동속도 220.0f를 440.0f로 변경함으로써 캐릭터 이동속도를 2배 증가시켰다.

따라서 기존 방어는 특정 함수의 코드 변조만 검사하고, 다른 함수의 코드 변조 및 비인가 DLL 로드를 충분히 탐지하지 못한다는 한계가 확인되었다.

방어 후보	장점 / 한계 / 선택 근거
DLL 로드 모듈 검사	비인가 DLL 탐지 가능 / DLL 은닉 기법에 취약
메모리·함수 무결성 검사	Hero 초기화 코드 변조 탐지 가능 / 검사 범위 확대에 따른 성능 부담
동작·상태 재검증	비정상 이동속도 탐지 가능 / 정상적인 속도 증가 효과와 구분 필요
최종 선택	함수 무결성 검사 + dll무결성 검사를 통한 비인가 dll탐지 + 동작 상태 재검증을 하기로 결정했다.

5. 방어 구현 및 정상 동작 확인

수정 파일 / 함수	변경 사항
IntegrityGuard.cpp	기존에는 GameWorld::Update에 대해서만 crc32검증만 하고 끝냈다면 이제는 Hero class초기화에 관련된 영역에 대해서도 같이 검증을 하고 앞으로의 확장성을 위해서 이 검증을 공통 검증 함수에게 맡겼다. 추가로 dll의 이름과 경로를 저장할 자료구조 그리고 현재 dll목록을 얻기 위한 api들을 추가했다.
IntergirityGuard.h	cpp변경에 따라서 새로 생긴 정규화 함수, dll탐색함수를 선언했다.
Appliacation.cpp	이러한 검증을 매번 업데이트마다 하기에는 오버헤드가 너무커서 그냥 app클래스에 검증을 넣어서 1초마다 검증하게 해서 오버헤드를 줄였습니다.
GameWorld.cpp	기존 파일에서 사용자가 이동한 다음에 시간에 따른 이동 거리가 초기 설정값의 1.1배를 넘는지 확인하고 넘는 경우에는 게임을 강제 종료시키는 코드를 추가했습니다.

검증	결과	비고

빌드·기존 테스트	PASS	없음
게임 기본 기능 / 오탐	정상	없음
성능 영향	없음	없음

6. 동일 공격 재시도 및 결론

항목	방어 적용 전	방어 적용 후
동일 DLL 주입 여부	o	o
변조 목표 달성 여부	o	x
방어 탐지·차단 여부	x	o

최종 결론 (공격 성패, 방어 효과, 우회 가능성, 다음 개선점)

공격과 공격에 대한 방어 둘 다 성공을 하긴 했지만 여전히 이를 우회할 방법은 많다. 그 중 하나는 다른 dll의 base파일을 바꾼다던지 아니면 intergrity부분을 아예 날려버린다던지의 방법이 있다.